

National University of Computer and Emerging Sciences



Summer 2026

Deadline: [13-july-2026]

Total Marks: 50

INSTRUCTIONS:

Please read the following instructions carefully before attempting the assignment.

1. This assignment covers material up to and including Operator Overloading (Weeks 1–9). No inheritance, polymorphism, or later topics are required or expected.
2. Submit one separate .cpp file per task. Format: 23L-XXXX-TASKX.cpp
3. Submit your assignment on Google Classroom / LMS by the given deadline. Late submission policy will follow the course outline.

4. Do not submit compressed files.
5. Your code must be properly commented and indented.
6. This is an individual assignment. Any form of plagiarism (from the internet, AI tools, or peers) will be dealt with as per department policy and may result in an F grade.
7. All dynamic memory must be properly allocated and deallocated. There must be no memory leaks or dangling pointers in any task.
8. Where arrays are used internally, prefer offset/pointer notation (`*(ptr + i)`) over subscript notation, consistent with course practice.
9. You are not allowed to use STL containers (e.g., `std::string`, `std::vector`) unless a task explicitly says otherwise. Build the required functionality yourself using pointers.
10. Make proper, well-named functions. Your `main()` should primarily be a sequence of function/test calls that demonstrate correct behavior.

Task 1: Fraction Calculator [10 Marks]

A local school wants a simple calculator app that lets students practice fraction arithmetic without converting to decimals. You are asked to design a `Fraction` class to power this feature.

Requirements:

- Design a class `Fraction` with private members `numerator` and `denominator` (both `int`).
- Provide a constructor with default values (`numerator = 0`, `denominator = 1`). If a `denominator` of 0 is passed in, default it to 1 instead.
- Overload the `+` operator (as a member function) to add two fractions and return the result as a new `Fraction` (do not reduce to lowest terms).
- Overload the `-` operator (as a member function) to subtract two fractions.
- Overload the `==` operator (as a member function) to check whether two fractions are equivalent, using cross-multiplication ($a/b == c/d$ if $a*d == c*b$) — do not convert to decimal.
- Overload the `<<` operator as a friend function so a `Fraction` can be printed directly in the form `a/b`.

Write a `main()` that creates at least two `Fraction` objects, demonstrates addition, subtraction, and an equivalence check, and prints the results using the overloaded `<<` operator.

Task 2: Playlist Manager [15 Marks]

A small music-streaming startup is prototyping a playlist feature. Playlists must be built using dynamically allocated memory only — no STL containers are allowed.

Requirements:

- Design a class Playlist that internally stores song titles as a dynamically allocated array of C-strings (char**), along with a capacity and a current song count.
- Provide a constructor Playlist(int capacity) that dynamically allocates the array of pointers.
- Provide an addSong(const char* title) function that dynamically allocates memory for the title and stores it in the next available slot. If the playlist is full, print a message and do nothing.
- Provide a copy constructor that performs a deep copy — a copied Playlist must not share any memory with the original.
- Overload the subscript operator [] to return the song title at a given index (read-only). Internally use pointer/offset notation.
- Overload the + operator as a friend (non-member) function that merges two playlists into a brand-new Playlist containing all songs from both, without modifying either original playlist.
- Provide a destructor that frees every dynamically allocated song title and the array itself — there must be no memory leaks.

Write a main() that builds two playlists, merges them using +, and prints every song in the merged playlist using the [] operator.

Task 3: Grayscale Matrix Processor [15 Marks]

A basic image-processing tool represents a grayscale image as a 2D grid of pixel intensities. You are asked to build the underlying Matrix class that will support combining and comparing images.

Requirements:

- Design a class Matrix with private members rows, cols, and a dynamically allocated 2D array data (int**), where every row is itself dynamically allocated.
- Provide a constructor Matrix(int rows, int cols) that dynamically allocates the full 2D structure.
- Provide setValue(int r, int c, int val) and getValue(int r, int c) helper functions, implemented internally using pointer/offset notation (*(*(data + r) + c)).
- Provide a copy constructor that performs a full deep copy of the 2D array (a copied Matrix must not share any row with the original).
- Overload the + operator (member function) for matrix addition. If dimensions do not match, print an error message and return an empty (0x0) Matrix instead of crashing.
- Overload the * operator (member function) for standard matrix multiplication. If the dimensions are incompatible (cols of the left-hand matrix $\neq$ rows of the right-hand matrix), print an error message and return an empty Matrix.
- Overload the == operator (member function) to check whether two matrices have identical dimensions and identical values in every cell.
- Overload the << operator as a friend function to print the matrix row by row.

- Provide a destructor that frees every row and the outer array — there must be no memory leaks or dangling pointers.

Write a main() that builds two 2x2 matrices, demonstrates +, *, and == on them, and prints all matrices using the overloaded << operator.

Task 4: Mini String Library [10 Marks]

A text-editor team needs a lightweight, custom string type for an upcoming Find & Replace feature. You may not use std::string or any STL container — build everything using a dynamically allocated char array.

Requirements:

- Design a class MyString with private members data (a dynamically allocated char*) and length (int).
- Provide a constructor MyString(const char* str = "") that dynamically allocates exactly enough memory and copies the characters in, using pointer/offset notation (no [] internally).
- Provide a copy constructor that performs a deep copy — a copied MyString must not share its character buffer with the original.
- Overload the subscript operator [] (member function) to return the character at a given index, implemented internally with pointer/offset notation.
- Overload the + operator (member function) to concatenate two MyString objects and return the result as a new MyString, without modifying either operand.
- Overload the == operator (member function) to compare two MyString objects character by character (not by comparing addresses).
- Overload the << operator as a friend function to print a MyString.
- Provide a destructor that frees the character buffer — there must be no memory leaks or dangling pointers.

Write a main() that builds at least two MyString objects, demonstrates concatenation with +, accesses a character with [], and checks equality with ==.

Good luck!